You can remove that `const` qualifier on **ToString** should you decide that you need to alter some object state in its implementation. But make each of your member functions either const or non-const, not both. In other words, don't overload an implementation function on `const`.

Aside from your implementation functions, another other place where const comes into the picture is in Windows Runtime function projections. Consider this code.

C++/WinRT

```
int main()
{
    winrt::Windows::Foundation::IStringable s{ winrt::make<MyStringable>()
};
    auto result{ s.ToString() };
}
```

For the call to **ToString** above, the **Go To Declaration** command in Visual Studio shows that the projection of the Windows Runtime **IStringable::ToString** into C++/WinRT looks like this.

C++/WinRT

```
winrt::hstring ToString() const;
```

Functions on the projection are const no matter how you choose to qualify your implementation of them. Behind the scenes, the projection calls the application binary interface (ABI), which amounts to a call through a COM interface pointer. The only state that the projected **ToString** interacts with is that COM interface pointer; and it certainly has no need to modify that pointer, so the function is const. This gives you the assurance that it won't change anything about the **IStringable** reference that you're calling through, and it ensures that you can call **ToString** even with a const reference to an **IStringable**.

Understand that these examples of `const` are implementation details of C++/WinRT projections and implementations; they constitute code hygiene for your benefit. There's no such thing as `const` on the COM nor Windows Runtime ABI (for member functions).

# Do you have any recommendations for decreasing the code size for C++/WinRT binaries?

When working with Windows Runtime objects, you should avoid the coding pattern shown below because it can have a negative impact on your application by causing more binary code than necessary to be generated.

C++/WinRT

```
anobject.b().c().d();
anobject.b().c().e();
anobject.b().c().f();
```

In the Windows Runtime world, the compiler is unable to cache either the value of `c()` or the interfaces for each method that's called through an indirection ('.'). Unless you intervene, that results in more virtual calls and reference counting overhead. The pattern above could easily generate twice as much code as strictly needed. Instead, prefer the pattern shown below wherever you can. It generates a lot less code, and it can also dramatically improve your run time performance.

C++/WinRT

```
auto a{ anobject.b().c() };
a.d();
a.e();
a.f();
```

The recommended pattern shown above applies not just to C++/WinRT but to all Windows Runtime language projections.

# How do I turn a string into a type (for navigation, for example)?

At the end of the Navigation view code example (which is mostly in C#), there's a C++/WinRT code snippet showing how to do this.

# How do I resolve ambiguities with GetCurrentTime and/or TRY?

The header file `winrt/Windows.UI.Xaml.Media.Animation.h` declares a method named **GetCurrentTime**, while `windows.h` (via `winbase.h`) defines a macro named **GetCurrentTime**. When the two collide, the C++ compiler produces "*error C4002: Too many arguments for function-like macro invocation GetCurrentTime*".

Similarly, `winrt/Windows.Globalization.h` declares a method named **TRY**, while `afx.h` defines a macro named **TRY**. When these collide, the C++ compiler produces "*error C2334: unexpected token(s) preceding '{'; skipping apparent function body*".

To remedy one or both issues, you can do this.

C++/WinRT

```
#pragma push_macro("GetCurrentTime")
#pragma push_macro("TRY")
#undef GetCurrentTime
#undef TRY
#include <winrt/include_your_cppwinrt_headers_here.h>
#include <winrt/include_your_cppwinrt_headers_here.h>
#pragma pop_macro("TRY")
#pragma pop_macro("GetCurrentTime")
```

# How do I speed up symbol loading?

In Visual Studio, **Tools** > **Options** > **Debugging** > **Symbols** > check *Load only specified modules*. You can then right-click DLLs in the stack list, and load individual modules.

> **Note**
>
> If this topic didn't answer your question, then you might find help by visiting the Visual Studio C++ developer community, or by using the c++-winrt tag on Stack Overflow.

# Troubleshooting C++/WinRT issues

Article • 10/27/2022

> **Note**
>
> For info about installing and using the **C++/WinRT** Visual Studio Extension (VSIX) (which provides project template support) see **Visual Studio support for C++/WinRT**.

This topic is up front so that you're aware of it right away; even if you don't need it yet. The table of troubleshooting symptoms and remedies below may be helpful to you whether you're cutting new code or porting an existing app. If you're porting, and you're eager to forge ahead and get to the stage where your project builds and runs, then you can make temporary progress by commenting or stubbing out any non-essential code that's causing issues, and then returning to pay off that debt later.

For a list of frequently-asked questions, see Frequently-asked questions.

## Tracking down XAML issues

XAML parse exceptions can be difficult to diagnose—particularly if there are no meaningful error messages within the exception. Make sure that the debugger is configured to catch first-chance exceptions (to try and catch the parsing exception early on). You may be able to inspect the exception variable in the debugger to determine whether the HRESULT or message has any useful information. Also, check Visual Studio's output window for error messages output by the XAML parser.

If your app terminates and all you know is that an unhandled exception was thrown during XAML markup parsing, then that could be the result of a reference (by key) to a missing resource. Or, it could be an exception thrown inside a UserControl, a custom control, or a custom layout panel. A last resort is a binary split. Remove about half of the markup from a XAML Page and re-run the app. You will then know whether the error is somewhere inside the half you removed (which you should now restore in any case) or in the half you did not remove. Repeat the process by splitting the half that contains the error, and so on, until you've zeroed in on the issue.

## Symptoms and remedies

| Symptom | Remedy |
| --- | --- |
| An exception is thrown at runtime with a HRESULT value of REGDB_E_CLASSNOTREGISTERED. | See Why am I getting a "class not registered" exception?. |
| The C++ compiler produces the error "'*implements_type*': *is not a member of any direct or indirect base class of* '<*projected type*>'". | This can happen when you call **make** with the namespace-unqualified name of your implementation type (**MyRuntimeClass**, for example), and you haven't included that type's header. The compiler interprets **MyRuntimeClass** as the projected type. The solution is to include the header for your implementation type (`MyRuntimeClass.h`, for example). |

| Symptom | Remedy |
| --- | --- |
| The C++ compiler produces the error "*attempting to reference a deleted function*". | This can happen when you call **make** and the implementation type that you pass as the template parameter has an `= delete` default constructor. Edit the implementation type's header file and change `= delete` to `= default`. You can also add a constructor into the IDL for the runtime class. |
| You've implemented **INotifyPropertyChanged**, but your XAML bindings are not updating (and the UI is not subscribing to **PropertyChanged**). | Remember to set `Mode=OneWay` (or TwoWay) on your binding expression in XAML markup. See XAML controls; bind to a C++/WinRT property. |
| You're binding a XAML items control to an observable collection, and an exception is thrown at runtime with the message "The parameter is incorrect". | In your IDL and your implementation, declare any observable collection as the type **Windows.Foundation.Collections.IVector<IInspectable>**. But return an object that implements **Windows.Foundation.Collections.IObservableVector<T>**, where T is your element type. See XAML items controls; bind to a C++/WinRT collection. |
| The C++ compiler produces an error of the form "'*MyImplementationType_base<MyImplementationType>': no appropriate default constructor available*". | This can happen when you have derived from a type that has a non-trivial constructor. Your derived type's constructor needs to pass along the parameters that the base type's constructor needs. For a worked example, see Deriving from a type that has a non-trivial constructor. |
| The C++ compiler produces the error "*cannot convert from 'const std::vector<std::wstring,std::allocator<_Ty>>' to 'const winrt::param::async_iterable<winrt::hstring> &*'". | This can happen when you pass a std::vector of std::wstring to a Windows Runtime API that expects a collection. For more info, see Standard C++ data types and C++/WinRT. |
| The C++ compiler produces the error "*cannot convert from 'const std::vector<winrt::hstring,std::allocator<_Ty>>' to 'const winrt::param::async_iterable<winrt::hstring> &*'". | This can happen when you pass a std::vector of winrt::hstring to an asynchronous Windows Runtime API that expects a collection, and you've neither copied nor moved the vector to the async callee. For more info, see Standard C++ data types and C++/WinRT. |
| When opening a project, Visual Studio produces the error "*The application for the project is not installed*". | If you haven't already, you need to install **Windows Universal tools for C++ development** from within Visual Studio's **New Project** dialog. If that doesn't resolve the issue, then the project may depend on the C++/WinRT Visual Studio Extension (VSIX) (see Visual Studio support for C++/WinRT. |
| The Windows App Certification Kit tests produce an error that one of your runtime classes "*does not derive from a Windows base class. All composable classes must ultimately derive from a type in the Windows namespace*". | Any runtime class (that you declare in your application) that derives from a base class is known as a *composable* class. The ultimate base class of a composable class must be a type originating in a Windows.* namespace; for example, **Windows.UI.Xaml.DependencyObject**. See XAML controls; bind to a C++/WinRT property for more details. |
| The C++ compiler produces a "*T must be WinRT type*" error for an EventHandler or TypedEventHandler delegate specialization. | Consider using **winrt::delegate<...T>** instead. See Author events in C++/WinRT. |

| Symptom | Remedy |
| --- | --- |
| The C++ compiler produces a "*T must be WinRT type*" error for a Windows Runtime asynchronous operation specialization. | Consider returning a Parallel Patterns Library (PPL) **task** instead. See Concurrency and asynchronous operations. |
| The C++ compiler produces a "*T must be WinRT type*" error when you call **winrt::xaml_typename**. | Use the projected type with **winrt::xaml_typename** (for example, use **BgLabelControlApp::BgLabelControl**), and not the implementation type(for example, don't use **BgLabelControlApp::implementation::BgLabelControl**). See XAML custom (templated) controls. |
| The C++ compiler produces "*error C2220: warning treated as error - no 'object' file generated*". | Either correct the warning, or set **C/C++** > **General** > **Treat Warnings As Errors** to **No (/WX-)**. |
| Your app crashes because an event handler in your C++/WinRT object is called after the object has been destroyed. | See Safely accessing the *this* pointer with an event-handling delegate. |
| The C++ compiler produces "*error C2338: This is only for weak ref support*". | You're requesting a weak reference for a type that passed the **winrt::no_weak_ref** marker struct as a template argument to its base class. See Opting out of weak reference support. |
| The C++ compiler produces "*consume_Something: function that returns 'auto' cannot be used before it is defined*" | See C3779: Why is the compiler giving me a "consume_Something: function that returns 'auto' cannot be used before it is defined" error?. |
| The C++ linker produces "*error LNK2019: Unresolved external symbol*" | See Why is the linker giving me a "LNK2019: Unresolved external symbol" error?. |
| The LLVM and Clang toolchain produces errors when used with C++/WinRT. | We don't support the LLVM and Clang toolchain for C++/WinRT, but if you wanted to emulate how we use it internally, then you could try an experiment such as the one described in Can I use LLVM/Clang to compile with C++/WinRT?. |
| The C++ compiler produces "*no appropriate default constructor available*" for a projected type. | If you're trying to delay the initialization of a runtime class object, or to consume and implement a runtime class in the same project, then you'll need to call the **std::nullptr_t** constructor. For more info, see Consume APIs with C++/WinRT. |
| The C++ compiler produces "*error C3861: 'from_abi': identifier not found*", and other errors originating in *base.h*. You may see this error if you are using Visual Studio 2017 (version 15.8.0 or later), and targeting the Windows SDK version 10.0.17134.0 (Windows 10, version 1803). | Either target a later (more conformant) version of the Windows SDK, or set project property **C/C++** > **Language** > **Conformance mode: No** (also, if **/permissive-** appears in project property **C/C++** > **Language** > **Command Line** under **Additional Options**, then delete it). |
| The C++ compiler produces "*error C2039: 'IUnknown': is not a member of '`global namespace'*". | See How to retarget your C++/WinRT project to a later version of the Windows SDK. |
| The C++ linker produces "*error LNK2019: unresolved external symbol _WINRT_CanUnloadNow@0 referenced in function _VSDesignerCanUnloadNow@0*" | See How to retarget your C++/WinRT project to a later version of the Windows SDK. |

| Symptom | Remedy |
| --- | --- |
| The build process produces the error message *The C++/WinRT VSIX no longer provides project build support. Please add a project reference to the Microsoft.Windows.CppWinRT Nuget package.* | Install the **Microsoft.Windows.CppWinRT** NuGet package into your project. For details, see Earlier versions of the VSIX extension. |
| The C++ linker produces *error LNK2019: unresolved external symbol*, with a mention of *winrt::impl::consume_Windows_Foundation_Collections_IVector*. | As of C++/WinRT 2.0, If you're using a range-based `for` on a Windows Runtime collection, then you'll now need to `#include <winrt/Windows.Foundation.Collections.h>`. |
| The C++ compiler produces "*error C4002: Too many arguments for function-like macro invocation GetCurrentTime*". | See How do I resolve ambiguities with GetCurrentTime and/or TRY?. |
| The C++ compiler produces "*error C2334: unexpected token(s) preceding '{'; skipping apparent function body*". | See How do I resolve ambiguities with GetCurrentTime and/or TRY?. |
| The C++ compiler produces "*winrt::impl::produce<D,I> cannot instantiate abstract class, due to missing GetBindingConnector*". | You need to `#include <winrt/Windows.UI.Xaml.Markup.h>`. |
| The C++ compiler produces "*error C2039: 'promise_type': is not a member of 'std::experimental::coroutine_traits<void>'*". | Your coroutine needs to return either an asynchronous operation object, or **winrt::fire_and_forget**. See Concurrency and asynchronous operations. |
| Your project produces "*ambiguous access of 'PopulatePropertyInfoOverride'*". | This error can occur when you declare one base class in your IDL and a different base class in your XAML markup. |
| Loading a C++/WinRT solution for the first time produces "*Designtime build failed for project 'MyProject.vcxproj' configuration 'Debug\|x86'. IntelliSense might be unavailable.*". | This IntelliSense issue will resolve after you build for the first time. |
| Attempting to specify **winrt::auto_revoke** when registering a delegate produces a **winrt::hresult_no_interface** exception. | See If your auto-revoke delegate fails to register. |
| In a C++/WinRT app, when consuming a C# Windows Runtime component that uses XAML, the compiler produces an error of the form "'*MyNamespace_XamlTypeInfo': is not a member of 'winrt::MyNamespace*'"—where *MyNamespace* is the name of the Windows Runtime component's namespace. | In `pch.h` in the consuming C++/WinRT app, add `#include <winrt/MyNamespace.MyNamespace_XamlTypeInfo.h>`—replacing *MyNamespace* as appropriate. |
| In a C++/WinRT project in Visual Studio, IntelliSense produces an error of the form "*error E1696: cannot open source file*". | Compile your newly created project at least once. Then right-click in the source code editor > **Rescan** > **Rescan File**. That will resolve all IntelliSense errors, including E1696. |

> **Note**
>
> If this topic didn't answer your question, then you might find help by visiting the **Visual Studio C++ developer community**, or by using the **c++-winrt tag on Stack Overflow**.

# Photo Editor C++/WinRT sample application

Article • 10/20/2022

> **Note**
>
> The sample is targeted and tested for Windows 10, version 1903 (10.0; Build 18362), and Visual Studio 2019. If you prefer, you can use project properties to retarget the project(s) to Windows 10, version 1809 (10.0; Build 17763), and/or open the sample with Visual Studio 2017.

To clone or download the sample application, see Photo Editor C++/WinRT sample application on the code samples gallery.

The Photo Editor application is a Universal Windows Platform (UWP) sample application that showcases development with the C++/WinRT language projection. The sample application allows you to retrieve photos from the **Pictures** library, and then edit the selected image with assorted photo effects. In the sample's source code, you'll see a number of common practices—such as data binding, and asynchronous actions and operations—performed using the C++/WinRT projection. Here are some of the specific features demonstrated by the sample.

- Use of Standard C++17 syntax and libraries with Windows Runtime (WinRT) APIs.
- Use of coroutines, including the use of co_await, co_return, **IAsyncAction**, and **IAsyncOperation<TResult>**.
- Creation and use of custom Windows Runtime class (runtime class) projected types and implementation types. For more info about these terms, see Consume APIs with C++/WinRT and Author APIs with C++/WinRT.
- Event handling, including the use of auto-revoking event tokens.
- Use of the external Win2D NuGet package, and Windows::UI::Composition, for image effects.
- XAML data binding, including the {x:Bind} markup extension.
- XAML styling and UI customization, including connected animations.

Also see Where can I find C++/WinRT sample apps?.

# String handling in C++/WinRT

Article • 10/20/2022

With C++/WinRT, you can call Windows Runtime APIs using C++ Standard Library wide string types such as **std::wstring** (note: not with narrow string types such as **std::string**). C++/WinRT does have a custom string type called **winrt::hstring** (defined in the C++/WinRT base library, which is `%WindowsSdkDir%Include\<WindowsTargetPlatformVersion>\cppwinrt\winrt\base.h`). And that's the string type that Windows Runtime constructors, functions, and properties actually take and return. But in many cases—thanks to **hstring**'s conversion constructors and conversion operators—you can choose whether or not to be aware of **hstring** in your client code. If you're *authoring* APIs, then you're more likely to need to know about **hstring**.

There are many string types in C++. Variants exist in many libraries in addition to **std::basic_string** from the C++ Standard Library. C++17 has string conversion utilities, and **std::basic_string_view**, to bridge the gaps between all of the string types. **winrt::hstring** provides convertibility with **std::wstring_view** to provide the interoperability that **std::basic_string_view** was designed for.

## Using std::wstring (and optionally winrt::hstring) with Uri

**Windows::Foundation::Uri** is constructed from a **winrt::hstring**.

C++/WinRT

```
public:
    Uri(winrt::hstring uri) const;
```

But **hstring** has conversion constructors that let you work with it without needing to be aware of it. Here's a code example showing how to make a **Uri** from a wide string literal, from a wide string view, and from a **std::wstring**.

C++/WinRT

```
#include <winrt/Windows.Foundation.h>
#include <string_view>

using namespace winrt;
using namespace Windows::Foundation;

int main()
```

```
{
    using namespace std::literals;

    winrt::init_apartment();

    // You can make a Uri from a wide string literal.
    Uri contosoUri{ L"http://www.contoso.com" };

    // Or from a wide string view.
    Uri contosoSVUri{ L"http://www.contoso.com"sv };

    // Or from a std::wstring.
    std::wstring wideString{ L"http://www.adventure-works.com" };
    Uri awUri{ wideString };
}
```

The property accessor **Uri::Domain** is of type **hstring**.

C++/WinRT

```
public:
    winrt::hstring Domain();
```

But, again, being aware of that detail is optional thanks to **hstring**'s conversion operator to **std::wstring_view**.

C++/WinRT

```
// Access a property of type hstring, via a conversion operator to a
standard type.
std::wstring domainWstring{ contosoUri.Domain() }; // L"contoso.com"
domainWstring = awUri.Domain(); // L"adventure-works.com"

// Or, you can choose to keep the hstring unconverted.
hstring domainHstring{ contosoUri.Domain() }; // L"contoso.com"
domainHstring = awUri.Domain(); // L"adventure-works.com"
```

Similarly, **IStringable::ToString** returns hstring.

C++/WinRT

```
public:
    hstring ToString() const;
```

**Uri** implements the **IStringable** interface.

C++/WinRT

```
// Access hstring's IStringable::ToString, via a conversion operator to a
standard type.
std::wstring tostringWstring{ contosoUri.ToString() }; //
L"http://www.contoso.com/"
tostringWstring = awUri.ToString(); // L"http://www.adventure-works.com/"

// Or you can choose to keep the hstring unconverted.
hstring tostringHstring{ contosoUri.ToString() }; //
L"http://www.contoso.com/"
tostringHstring = awUri.ToString(); // L"http://www.adventure-works.com/"
```

You can use the **hstring::c_str function** to get a standard wide string from an **hstring** (just as you can from a **std::wstring**).

C++/WinRT

```
#include <iostream>
std::wcout << tostringHstring.c_str() << std::endl;
```

If you have an **hstring** then you can make a **Uri** from it.

C++/WinRT

```
Uri awUriFromHstring{ tostringHstring };
```

Consider a method that takes an **hstring**.

C++/WinRT

```
public:
    Uri CombineUri(winrt::hstring relativeUri) const;
```

All of the options you've just seen also apply in such cases.

C++/WinRT

```
std::wstring contact{ L"contact" };
contosoUri = contosoUri.CombineUri(contact);

std::wcout << contosoUri.ToString().c_str() << std::endl;
```

**hstring** has a member **std::wstring_view** conversion operator, and the conversion is achieved at no cost.

C++/WinRT

```
void legacy_print(std::wstring_view view);

void Print(winrt::hstring const& hstring)
{
    legacy_print(hstring);
}
```

## winrt::hstring functions and operators

A host of constructors, operators, functions, and iterators are implemented for winrt::hstring.

An **hstring** is a range, so you can use it with range-based `for`, or with `std::for_each`. It also provides comparison operators for naturally and efficiently comparing against its counterparts in the C++ Standard Library. And it includes everything you need to use **hstring** as a key for associative containers.

We recognize that many C++ libraries use **std::string**, and work exclusively with UTF-8 text. As a convenience, we provide helpers, such as winrt::to_string and winrt::to_hstring, for converting back and forth.

`WINRT_ASSERT` is a macro definition, and it expands to _ASSERTE.

C++/WinRT

```
winrt::hstring w{ L"Hello, World!" };

std::string c = winrt::to_string(w);
WINRT_ASSERT(c == "Hello, World!");

w = winrt::to_hstring(c);
WINRT_ASSERT(w == L"Hello, World!");
```

For more examples and info about **hstring** functions and operators, see the winrt::hstring API reference topic.

## The rationale for winrt::hstring and winrt::param::hstring

The Windows Runtime is implemented in terms of **wchar_t** characters, but the Windows Runtime's Application Binary Interface (ABI) is not a subset of what either **std::wstring** or **std::wstring_view** provide. Using those would lead to significant inefficiency. Instead, C++/WinRT provides **winrt::hstring**, which represents an immutable string consistent

with the underlying HSTRING, and implemented behind an interface similar to that of **std::wstring**.

You may notice that C++/WinRT input parameters that should logically accept **winrt::hstring** actually expect **winrt::param::hstring**. The **param** namespace contains a set of types used exclusively to optimize input parameters to naturally bind to C++ Standard Library types and avoid copies and other inefficiencies. You shouldn't use these types directly. If you want to use an optimization for your own functions then use **std::wstring_view**. Also see Passing parameters into the ABI boundary.

The upshot is that you can largely ignore the specifics of Windows Runtime string management, and just work with efficiency with what you know. And that's important, given how heavily strings are used in the Windows Runtime.

## Formatting strings

One option for string-formatting is **std::wostringstream**. Here's an example that formats and displays a simple debug trace message.

C++/WinRT

```
#include <sstream>
#include <winrt/Windows.UI.Input.h>
#include <winrt/Windows.UI.Xaml.Input.h>
...
void
MainPage::OnPointerPressed(winrt::Windows::UI::Xaml::Input::PointerRoutedEve
ntArgs const& e)
{
    winrt::Windows::Foundation::Point const point{
e.GetCurrentPoint(nullptr).Position() };
    std::wostringstream wostringstream;
    wostringstream << L"Pointer pressed at (" << point.X << L"," << point.Y
<< L")" << std::endl;
    ::OutputDebugString(wostringstream.str().c_str());
}
```

## The correct way to set a property

You set a property by passing a value to a setter function. Here's an example.

C++/WinRT

```
// The right way to set the Text property.
myTextBlock.Text(L"Hello!");
```

The code below is incorrect. It compiles, but all it does is to modify the temporary **winrt::hstring** returned by the **Text()** accessor function, and then to throw the result away.

C++/WinRT

```
// *Not* the right way to set the Text property.
myTextBlock.Text() = L"Hello!";
```

# Important APIs

- winrt::hstring struct
- winrt::to_hstring function
- winrt::to_string function

# Standard C++ data types and C++/WinRT

Article • 10/20/2022

With C++/WinRT, you can call Windows Runtime APIs using Standard C++ data types, including some C++ Standard Library data types. You can pass standard strings to APIs (see String handling in C++/WinRT), and you can pass initializer lists and standard containers to APIs that expect a semantically equivalent collection.

Also see Passing parameters into the ABI boundary.

## Standard initializer lists

An initializer list (**std::initializer_list**) is a C++ Standard Library construct. You can use initializer lists when you call certain Windows Runtime constructors and methods. For example, you can call **DataWriter::WriteBytes** with one.

C++/WinRT

```
#include <winrt/Windows.Storage.Streams.h>

using namespace winrt::Windows::Storage::Streams;

int main()
{
    winrt::init_apartment();

    InMemoryRandomAccessStream stream;
    DataWriter dataWriter{stream};
    dataWriter.WriteBytes({ 99, 98, 97 }); // the initializer list is
converted to a winrt::array_view before being passed to WriteBytes.
}
```

There are two pieces involved in making this work. First, the **DataWriter::WriteBytes** method takes a parameter of type **winrt::array_view**.

C++/WinRT

```
void WriteBytes(winrt::array_view<uint8_t const> value) const
```

**winrt::array_view** is a custom C++/WinRT type that safely represents a contiguous series of values (it is defined in the C++/WinRT base library, which is `%WindowsSdkDir%Include\<WindowsTargetPlatformVersion>\cppwinrt\winrt\base.h`).

Second, **winrt::array_view** has an initializer-list constructor.

C++/WinRT

```
template <typename T> winrt::array_view(std::initializer_list<T> value)
noexcept
```

In many cases, you can choose whether or not to be aware of **winrt::array_view** in your programming. If you choose *not* to be aware of it then you won't have any code to change if and when an equivalent type appears in the C++ Standard Library.

You can pass an initializer list to a Windows Runtime API that expects a collection parameter. Take **StorageItemContentProperties::RetrievePropertiesAsync** for example.

C++/WinRT

```
IAsyncOperation<IMap<winrt::hstring, IInspectable>>
StorageItemContentProperties::RetrievePropertiesAsync(IIterable<winrt::hstri
ng> propertiesToRetrieve) const;
```

You can call that API with an initializer list like this.

C++/WinRT

```
IAsyncAction retrieve_properties_async(StorageFile const storageFile)
{
    auto properties{ co_await
storageFile.Properties().RetrievePropertiesAsync({ L"System.ItemUrl" }) };
}
```

Two factors are at work here. First, the callee constructs a **std::vector** from the initializer list (this callee is asynchronous, so it's able to own that object, which it must). Second, C++/WinRT transparently (and without introducing copies) binds **std::vector** as a Windows Runtime collection parameter.

# Standard arrays and vectors

winrt::array_view also has conversion constructors from **std::vector** and **std::array**.

C++/WinRT

```
template <typename C, size_type N> winrt::array_view(std::array<C, N>&
value) noexcept
template <typename C> winrt::array_view(std::vector<C>& vectorValue)
noexcept
```

So, you could instead call **DataWriter::WriteBytes** with a **std::vector**.

C++/WinRT

```
std::vector<byte> theVector{ 99, 98, 97 };
dataWriter.WriteBytes(theVector); // theVector is converted to a
winrt::array_view before being passed to WriteBytes.
```

Or with a **std::array**.

C++/WinRT

```
std::array<byte, 3> theArray{ 99, 98, 97 };
dataWriter.WriteBytes(theArray); // theArray is converted to a
winrt::array_view before being passed to WriteBytes.
```

C++/WinRT binds **std::vector** as a Windows Runtime collection parameter. So, you can pass a **std::vector<winrt::hstring>**, and it will be converted to the appropriate Windows Runtime collection of **winrt::hstring**. There's an extra detail to bear in mind if the callee is asynchronous. Due to the implementation details of that case, you'll need to provide an rvalue, so you must provide a copy or a move of the vector. In the code example below, we move ownership of the vector to the object of the parameter type accepted by the async callee (and then we're careful not to access `vecH` again after moving it). If you want to know more about rvalues, see Value categories, and references to them.

C++/WinRT

```
IAsyncAction retrieve_properties_async(StorageFile const storageFile,
std::vector<winrt::hstring> vecH)
{
    auto properties{ co_await
storageFile.Properties().RetrievePropertiesAsync(std::move(vecH)) };
}
```

But you can't pass a **std::vector<std::wstring>** where a Windows Runtime collection is expected. This is because, having converted to the appropriate Windows Runtime collection of **std::wstring**, the C++ language won't then coerce that collection's type parameter(s). Consequently, the following code example won't compile (and the solution is to pass a **std::vector<winrt::hstring>** instead, as shown above).

C++/WinRT

```
IAsyncAction retrieve_properties_async(StorageFile const storageFile,
std::vector<std::wstring> vecW)
```

```
{
    auto properties{ co_await
storageFile.Properties().RetrievePropertiesAsync(std::move(vecW)) }; //
error! Can't convert from vector of wstring to async_iterable of hstring.
}
```

## Raw arrays, and pointer ranges

Bearing in mind the caveat that an equivalent type may exist in the future in the C++ Standard Library, you can also work directly with **winrt::array_view** if you choose to, or need to.

**winrt::array_view** has conversion constructors from a raw array, and from a range of **T*** (pointers to the element type).

C++/WinRT

```
using namespace winrt;
...
byte theRawArray[]{ 99, 98, 97 };
array_view<byte const> fromRawArray{ theRawArray };
dataWriter.WriteBytes(fromRawArray); // the winrt::array_view is passed to
WriteBytes.

array_view<byte const> fromRange{ theArray.data(), theArray.data() + 2 }; //
just the first two elements.
dataWriter.WriteBytes(fromRange); // the winrt::array_view is passed to
WriteBytes.
```

## winrt::array_view functions and operators

A host of constructors, operators, functions, and iterators are implemented for **winrt::array_view**. A **winrt::array_view** is a range, so you can use it with range-based `for`, or with **std::for_each**.

For more examples and info, see the winrt::array_view API reference topic.

## IVector<T> and standard iteration constructs

SyndicationFeed.Items is an example of a Windows Runtime API that returns a collection of type IVector<T> (projected into C++/WinRT as **winrt::Windows::Foundation::Collections::IVector<T>**). You can use this type with standard iteration constructs, such as range-based `for`.

C++/WinRT

```
// main.cpp
#include "pch.h"
#include <winrt/Windows.Web.Syndication.h>
#include <iostream>

using namespace winrt;
using namespace Windows::Web::Syndication;

void PrintFeed(SyndicationFeed const& syndicationFeed)
{
    for (SyndicationItem const& syndicationItem : syndicationFeed.Items())
    {
        std::wcout << syndicationItem.Title().Text().c_str() << std::endl;
    }
}
```

## C++ coroutines with asynchronous Windows Runtime APIs

You can continue to use the Parallel Patterns Library (PPL) when calling asynchronous Windows Runtime APIs. However, in many cases, C++ coroutines provide an efficient and more easily-coded idiom for interacting with asynchronous objects. For more info, and code examples, see Concurrency and asynchronous operations with C++/WinRT.

## Important APIs

- IVector<T> interface
- winrt::array_view struct template

## Related topics

- String handling in C++/WinRT

# Boxing and unboxing values to IInspectable with C++/WinRT

Article • 10/20/2022

> **Note**
>
> You can box and unbox not only scalar values, but also most kinds of arrays (with the exception of arrays of enumerations) by using the **winrt::box_value** and **winrt::unbox_value** functions. You can unbox only scalar values by using the **winrt::unbox_value_or** function.

The **IInspectable interface** is the root interface of every runtime class in the Windows Runtime (WinRT). This is an analogous idea to **IUnknown** being at the root of every COM interface and class; and **System.Object** being at the root of every Common Type System class.

In other words, a function that expects **IInspectable** can be passed an instance of any runtime class. But you can't directly pass to such a function a scalar value (such as a numeric or text value), nor an array. Instead, a scalar or array value needs to be wrapped inside a reference class object. That wrapping process is known as *boxing* the value.

> **Important**
>
> You can box and unbox any type that you can pass to a Windows Runtime API. In other words, a Windows Runtime type. Numeric and text values (strings), and arrays, are some examples given above. Another example is a `struct` that you define in IDL. If you try to box a regular C++ `struct` (one that's not defined in IDL), then the compiler will remind you that you can box only a Windows Runtime type. A runtime class is a Windows Runtime type, but you can of course pass runtime classes to Windows Runtime APIs without boxing them.

C++/WinRT provides the **winrt::box_value** function, which takes a scalar or array value, and returns the value boxed into an **IInspectable**. For unboxing an **IInspectable** back into a scalar or array value, there is the **winrt::unbox_value** function. For unboxing an **IInspectable** back into a scalar value, there is also the **winrt::unbox_value_or** function.

## Examples of boxing a value

The LaunchActivatedEventArgs::Arguments accessor function returns a winrt::hstring, which is a scalar value. We can box that **hstring** value and pass it to a function that expects **IInspectable** like this.

C++/WinRT

```
void App::OnLaunched(LaunchActivatedEventArgs const& e)
{
    ...
    rootFrame.Navigate(winrt::xaml_typename<BlankApp1::MainPage>(),
winrt::box_value(e.Arguments()));
    ...
}
```

To set the content property of a XAML Button, you call the Button::Content mutator function. To set the content property to a string value, you can use this code.

C++/WinRT

```
Button().Content(winrt::box_value(L"Clicked"));
```

First, the hstring conversion constructor converts the string literal into an **hstring**. Then the overload of **winrt::box_value** that takes an **hstring** is invoked.

# Examples of unboxing an IInspectable

In your own functions that expect **IInspectable**, you can use winrt::unbox_value to unbox, and you can use winrt::unbox_value_or to unbox with a default value. You can also use try_as to unbox to a **std::optional**.

C++/WinRT

```
void Unbox(winrt::Windows::Foundation::IInspectable const& object)
{
    hstring hstringValue = unbox_value<hstring>(object); // Throws if object
is not a boxed string.
    hstringValue = unbox_value_or<hstring>(object, L"Default"); // Returns
L"Default" if object is not a boxed string.
    float floatValue = unbox_value_or<float>(object, 0.f); // Returns 0.0 if
object is not a boxed float.
    std::optional<int> optionalInt = object.try_as<int>(); // Returns
std::nullopt if object is not a boxed int.
}
```

## Determine the type of a boxed value

If you receive a boxed value and you're unsure what type it contains (you need to know its type in order to unbox it), then you can query the boxed value for its **IPropertyValue** interface, and then call **Type** on that. Here's a code example.

`WINRT_ASSERT` is a macro definition, and it expands to _ASSERTE.

C++/WinRT

```
float pi = 3.14f;
auto piInspectable = winrt::box_value(pi);
auto piPropertyValue =
piInspectable.as<winrt::Windows::Foundation::IPropertyValue>();
WINRT_ASSERT(piPropertyValue.Type() ==
winrt::Windows::Foundation::PropertyType::Single);
```

# Important APIs

- IInspectable interface
- winrt::box_value function template
- winrt::hstring struct
- winrt::unbox_value function template
- winrt::unbox_value_or function template

# Consume APIs with C++/WinRT

Article • 04/02/2024

This topic shows how to consume C++/WinRT APIs, whether they're part of Windows, implemented by a third-party component vendor, or implemented by yourself.

> **ⓘ Important**
>
> So that the code examples in this topic are short, and easy for you to try out, you can reproduce them by creating a new **Windows Console Application (C++/WinRT)** project, and copy-pasting code. However, you can't consume arbitrary custom (third-party) Windows Runtime types from an unpackaged app like that. You can consume only Windows types that way.
>
> To consume custom (third-party) Windows Runtime types from a console app, you'll need to give the app a *package identity* so that it can resolve the consumed custom types' registration. For more info, see Windows Application Packaging Project.
>
> Alternatively, create a new project from the **Blank App (C++/WinRT)**, **Core App (C++/WinRT)**, or **Windows Runtime Component (C++/WinRT)** project templates. Those app types already have a *package identity*.

## If the API is in a Windows namespace

This is the most common case in which you'll consume a Windows Runtime API. For every type in a Windows namespace defined in metadata, C++/WinRT defines a C++-friendly equivalent (called the *projected type*). A projected type has the same fully-qualified name as the Windows type, but it's placed in the C++ **winrt** namespace using C++ syntax. For example, **Windows::Foundation::Uri** is projected into C++/WinRT as **winrt::Windows::Foundation::Uri**.

Here's a simple code example. If you want to copy-paste the following code examples directly into the main source code file of a **Windows Console Application (C++/WinRT)** project, then first set **Not Using Precompiled Headers** in project properties.

C++/WinRT

```
// main.cpp
#include <winrt/Windows.Foundation.h>
```

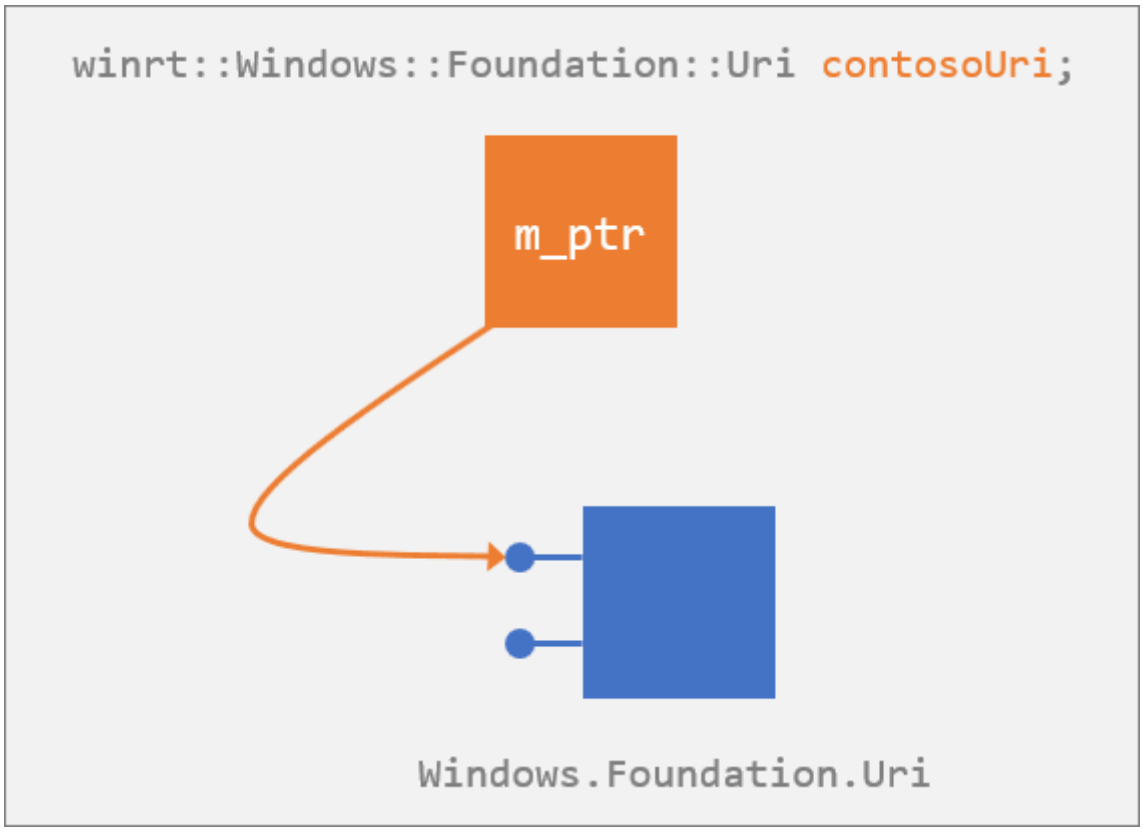
```
using namespace winrt;
using namespace Windows::Foundation;

int main()
{
    winrt::init_apartment();
    Uri contosoUri{ L"http://www.contoso.com" };
    Uri combinedUri = contosoUri.CombineUri(L"products");
}
```

The included header `winrt/Windows.Foundation.h` is part of the SDK, found inside the folder `%WindowsSdkDir%Include<WindowsTargetPlatformVersion>\cppwinrt\winrt\`. The headers in that folder contain Windows namespace types projected into C++/WinRT. In this example, `winrt/Windows.Foundation.h` contains **winrt::Windows::Foundation::Uri**, which is the projected type for the runtime class **Windows::Foundation::Uri**.

> **Tip**
>
> Whenever you want to use a type from a Windows namespace, include the C++/WinRT header corresponding to that namespace. The `using namespace` directives are optional, but convenient.

In the code example above, after initializing C++/WinRT, we stack-allocate a value of the **winrt::Windows::Foundation::Uri** projected type via one of its publicly documented constructors (**Uri(String)**, in this example). For this, the most common use case, that's typically all you have to do. Once you have a C++/WinRT projected type value, you can treat it as if it were an instance of the actual Windows Runtime type, since it has all the same members.

In fact, that projected value is a proxy; it's essentially just a smart pointer to a backing object. The projected value's constructor(s) call **RoActivateInstance** to create an instance of the backing Windows Runtime class (**Windows.Foundation.Uri**, in this case), and store that object's default interface inside the new projected value. As illustrated below, your calls to the projected value's members actually delegate, via the smart pointer, to the backing object; which is where state changes occur.




When the `contosoUri` value falls out of scope, it destructs, and releases its reference to the default interface. If that reference is the last reference to the backing Windows Runtime **Windows.Foundation.Uri** object, the backing object destructs as well.

> **Tip**
>
> A *projected type* is a wrapper over a Windows Runtime type for purposes of consuming its APIs. For example, a *projected interface* is a wrapper over a Windows Runtime interface.

## C++/WinRT projection headers

To consume Windows namespace APIs from C++/WinRT, you include headers from the `%WindowsSdkDir%Include<WindowsTargetPlatformVersion>\cppwinrt\winrt` folder. You must include the headers corresponding to each namespace you use.

For example, for the **Windows::Security::Cryptography::Certificates** namespace, the equivalent C++/WinRT type definitions reside in `winrt/Windows.Security.Cryptography.Certificates.h`. Including that header gives you access to all the types in the **Windows::Security::Cryptography::Certificates** namespace.

Sometimes, one namespace header will include portions of related namespace headers, but you shouldn't rely on this implementation detail. Explicitly include the headers for

the namespaces you use.

For example, the Certificate::GetCertificateBlob method returns an Windows::Storage::Streams::IBuffer interface. Before calling the Certificate::GetCertificateBlob method, you must include the `winrt/Windows.Storage.Streams.h` namespace header file to ensure that you can receive and operate on the returned Windows::Storage::Streams::IBuffer.

Forgetting to include the required namespace headers before using types in that namespace is a common source of build errors.

## Accessing members via the object, via an interface, or via the ABI

With the C++/WinRT projection, the runtime representation of a Windows Runtime class is no more than the underlying ABI interfaces. But, for your convenience, you can code against classes in the way that their author intended. For example, you can call the **ToString** method of a Uri as if that were a method of the class (in fact, under the covers, it's a method on the separate **IStringable** interface).

`WINRT_ASSERT` is a macro definition, and it expands to _ASSERTE.

C++/WinRT

```
Uri contosoUri{ L"http://www.contoso.com" };
WINRT_ASSERT(contosoUri.ToString() == L"http://www.contoso.com/"); //
QueryInterface is called at this point.
```

This convenience is achieved via a query for the appropriate interface. But you're always in control. You can opt to give away a little of that convenience for a little performance by retrieving the IStringable interface yourself and using it directly. In the code example below, you obtain an actual IStringable interface pointer at run time (via a one-time query). After that, your call to **ToString** is direct, and avoids any further call to QueryInterface.

C++/WinRT

```
...
IStringable stringable = contosoUri; // One-off QueryInterface.
WINRT_ASSERT(stringable.ToString() == L"http://www.contoso.com/");
```

You might choose this technique if you know you'll be calling several methods on the same interface.

Incidentally, if you do want to access members at the ABI level then you can. The code example below shows how, and there are more details and code examples in Interop between C++/WinRT and the ABI.

C++/WinRT

```
#include <Windows.Foundation.h>
#include <unknwn.h>
#include <winrt/Windows.Foundation.h>
using namespace winrt::Windows::Foundation;

int main()
{
    winrt::init_apartment();
    Uri contosoUri{ L"http://www.contoso.com" };

    int port{ contosoUri.Port() }; // Access the Port "property" accessor
via C++/WinRT.

    winrt::com_ptr<ABI::Windows::Foundation::IUriRuntimeClass> abiUri{
        contosoUri.as<ABI::Windows::Foundation::IUriRuntimeClass>() };
    HRESULT hr = abiUri->get_Port(&port); // Access the get_Port ABI
function.
}
```

# Delayed initialization

In C++/WinRT, each projected type has a special C++/WinRT **std::nullptr_t** constructor. With the exception of that one, all projected-type constructors—including the default constructor—cause a backing Windows Runtime object to be created, and give you a smart pointer to it. So, that rule applies anywhere that the default constructor is used, such as uninitialized local variables, uninitialized global variables, and uninitialized member variables.

If, on the other hand, you want to construct a variable of a projected type without it in turn constructing a backing Windows Runtime object (so that you can delay that work until later), then you can do that. Declare your variable or field using that special C++/WinRT **std::nullptr_t** constructor (which the C++/WinRT projection injects into every runtime class). We use that special constructor with *m_gamerPicBuffer* in the code example below.

C++/WinRT

```
#include <winrt/Windows.Storage.Streams.h>
using namespace winrt::Windows::Storage::Streams;

#define MAX_IMAGE_SIZE 1024

struct Sample
{
    void DelayedInit()
    {
        // Allocate the actual buffer.
        m_gamerPicBuffer = Buffer(MAX_IMAGE_SIZE);
    }

private:
    Buffer m_gamerPicBuffer{ nullptr };
};

int main()
{
    winrt::init_apartment();
    Sample s;
    // ...
    s.DelayedInit();
}
```

All constructors on the projected type *except* the **std::nullptr_t** constructor cause a backing Windows Runtime object to be created. The **std::nullptr_t** constructor is essentially a no-op. It expects the projected object to be initialized at a subsequent time. So, whether a runtime class has a default constructor or not, you can use this technique for efficient delayed initialization.

This consideration affects other places where you're invoking the default constructor, such as in vectors and maps. Consider this code example, for which you'll need a **Blank App (C++/WinRT)** project.

C++/WinRT

```
std::map<int, TextBlock> lookup;
lookup[2] = value;
```

The assignment creates a new **TextBlock**, and then immediately overwrites it with `value`. Here's the remedy.

C++/WinRT

```
std::map<int, TextBlock> lookup;
lookup.insert_or_assign(2, value);
```

Also see How the default constructor affects collections.

## Don't delay-initialize by mistake

Be careful that you don't invoke the **std::nullptr_t** constructor by mistake. The compiler's conflict resolution favors it over the factory constructors. For example, consider these two runtime class definitions.

idl

```
// GiftBox.idl
runtimeclass GiftBox
{
    GiftBox();
}

// Gift.idl
runtimeclass Gift
{
    Gift(GiftBox giftBox); // You can create a gift inside a box.
}
```

Let's say that we want to construct a **Gift** that isn't inside a box (a **Gift** that's constructed with an uninitialized **GiftBox**). First, let's look at the *wrong* way to do that. We know that there'a **Gift** constructor that takes a **GiftBox**. But if we're tempted to pass a null **GiftBox** (invoking the **Gift** constructor via uniform initialization, as we do below), then we *won't* get the result we want.

C++/WinRT

```
// These are *not* what you intended. Doing it in one of these two ways
// actually *doesn't* create the intended backing Windows Runtime Gift
object;
// only an empty smart pointer.

Gift gift{ nullptr };
auto gift{ Gift(nullptr) };
```

What you get here is an uninitialized **Gift**. You don't get a **Gift** with an uninitialized **GiftBox**. Here's the *correct* way to do that.

C++/WinRT

```
// Doing it in one of these two ways creates an initialized
// Gift with an uninitialized GiftBox.
```

```
Gift gift{ GiftBox{ nullptr } };
auto gift{ Gift(GiftBox{ nullptr }) };
```

In the incorrect example, passing a `nullptr` literal resolves in favor of the delay-initializing constructor. To resolve in favor of the factory constructor, the type of the parameter must be a **GiftBox**. You still have the option to pass an explicitly delay-initializing **GiftBox**, as shown in the correct example.

This next example is *also* correct, because the parameter has type GiftBox, and not **std::nullptr_t**.

C++/WinRT

```
GiftBox giftBox{ nullptr };
Gift gift{ giftBox }; // Calls factory constructor.
```

It's only when you pass a `nullptr` literal that the ambiguity arises.

# Don't copy-construct by mistake.

This caution is similar to the one described in the Don't delay-initialize by mistake section above.

In addition to the delay-initializing constructor, the C++/WinRT projection also injects a copy constructor into every runtime class. It's a single-parameter constructor that accepts the same type as the object being constructed. The resulting smart pointer points to the same backing Windows Runtime object as that pointed to by its constructor parameter. The result is two smart pointer objects pointing to the same backing object.

Here's a runtime class definition that we'll use in the code examples.

idl

```
// GiftBox.idl
runtimeclass GiftBox
{
    GiftBox(GiftBox biggerBox); // You can place a box inside a bigger box.
}
```

Let's say that we want to construct a **GiftBox** inside a larger **GiftBox**.

C++/WinRT

```
GiftBox bigBox{ ... };

// These are *not* what you intended. Doing it in one of these two ways
// copies bigBox's backing-object-pointer into smallBox.
// The result is that smallBox == bigBox.

GiftBox smallBox{ bigBox };
auto smallBox{ GiftBox(bigBox) };
```

The *correct* way to do it is to call the activation factory explicitly.

C++/WinRT

```
GiftBox bigBox{ ... };

// These two ways call the activation factory explicitly.

GiftBox smallBox{
    winrt::get_activation_factory<GiftBox, IGiftBoxFactory>
().CreateInstance(bigBox) };
auto smallBox{
    winrt::get_activation_factory<GiftBox, IGiftBoxFactory>
().CreateInstance(bigBox) };
```

## If the API is implemented in a Windows Runtime component

This section applies whether you authored the component yourself, or it came from a vendor.

> **Note**
>
> For info about installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support), see Visual Studio support for C++/WinRT.

In your application project, reference the Windows Runtime component's Windows Runtime metadata (`.winmd`) file, and build. During the build, the `cppwinrt.exe` tool generates a standard C++ library that fully describes—or *projects*—the API surface for the component. In other words, the generated library contains the projected types for the component.

Then, just as for a Windows namespace type, you include a header and construct the projected type via one of its constructors. Your application project's startup code registers the runtime class, and the projected type's constructor calls **RoActivateInstance** to activate the runtime class from the referenced component.

C++/WinRT

```
#include <winrt/ThermometerWRC.h>

struct App : implements<App, IFrameworkViewSource, IFrameworkView>
{
    ThermometerWRC::Thermometer thermometer;
    ...
};
```

For more details, code, and a walkthrough of consuming APIs implemented in a Windows Runtime component, see Windows Runtime components with C++/WinRT and Author events in C++/WinRT.

# If the API is implemented in the consuming project

The code example in this section is taken from the topic XAML controls; bind to a C++/WinRT property. See that topic for more details, code, and a walkthrough of consuming a runtime class that's implemented in the same project that consumes it.

A type that's consumed from XAML UI must be a runtime class, even if it's in the same project as the XAML. For this scenario, you generate a projected type from the runtime class's Windows Runtime metadata (`.winmd`). Again, you include a header, but then you have a choice between the C++/WinRT version 1.0 or version 2.0 ways of constructing the instance of the runtime class. The version 1.0 method uses **winrt::make**; the version 2.0 method is known as *uniform construction*. Let's look at each in turn.

## Constructing by using winrt::make

Let's start with the default (C++/WinRT version 1.0) method, because it's a good idea to be at least familiar with that pattern. You construct the projected type via its **std::nullptr_t** constructor. That constructor doesn't perform any initialization, so you must next assign a value to the instance via the **winrt::make** helper function, passing any necessary constructor arguments. A runtime class implemented in the same project as the consuming code doesn't need to be registered, nor instantiated via Windows Runtime/COM activation.

See XAML controls; bind to a C++/WinRT property for a full walkthrough. This section shows extracts from that walkthrough.

C++/WinRT

```
// MainPage.idl
import "BookstoreViewModel.idl";
namespace Bookstore
{
    runtimeclass MainPage : Windows.UI.Xaml.Controls.Page
    {
        BookstoreViewModel MainViewModel{ get; };
    }
}

// MainPage.h
...
struct MainPage : MainPageT<MainPage>
{
    ...
    private:
        Bookstore::BookstoreViewModel m_mainViewModel{ nullptr };
};
...

// MainPage.cpp
...
#include "BookstoreViewModel.h"

MainPage::MainPage()
{
    m_mainViewModel =
winrt::make<Bookstore::implementation::BookstoreViewModel>();
    ...
}
```

## Uniform construction

With C++/WinRT version 2.0 and later, there's an optimized form of construction available to you known as *uniform construction* (see News, and changes, in C++/WinRT 2.0).

See XAML controls; bind to a C++/WinRT property for a full walkthrough. This section shows extracts from that walkthrough.

To use uniform construction instead of **winrt::make**, you'll need an activation factory. A good way to generate one is to add a constructor to your IDL.

idl

```
// MainPage.idl
import "BookstoreViewModel.idl";
namespace Bookstore
{
    runtimeclass MainPage : Windows.UI.Xaml.Controls.Page
    {
        MainPage();
        BookstoreViewModel MainViewModel{ get; };
    }
}
```

Then, in `MainPage.h` declare and initialize *m_mainViewModel* in just one step, as shown below.

C++/WinRT

```
// MainPage.h
...
struct MainPage : MainPageT<MainPage>
{
    ...
    private:
        Bookstore::BookstoreViewModel m_mainViewModel;
        ...
    };
}
...
```

And then, in the **MainPage** constructor in `MainPage.cpp`, there's no need for the code `m_mainViewModel = winrt::make<Bookstore::implementation::BookstoreViewModel>();`.

For more info about uniform construction, and code examples, see Opt in to uniform construction, and direct implementation access.

# Instantiating and returning projected types and interfaces

Here's an example of what projected types and interfaces might look like in your consuming project. Remember that a projected type (such as the one in this example), is tool-generated, and is not something that you'd author yourself.

C++/WinRT

```
struct MyRuntimeClass : MyProject::IMyRuntimeClass,
impl::require<MyRuntimeClass,
```

```
    Windows::Foundation::IStringable, Windows::Foundation::IClosable>
```

**MyRuntimeClass** is a projected type; projected interfaces include **IMyRuntimeClass**, **IStringable**, and **IClosable**. This topic has shown the different ways in which you can instantiate a projected type. Here's a reminder and summary, using **MyRuntimeClass** as an example.

C++/WinRT

```
// The runtime class is implemented in another compilation unit (it's either
a Windows API,
// or it's implemented in a second- or third-party component).
MyProject::MyRuntimeClass myrc1;

// The runtime class is implemented in the same compilation unit.
MyProject::MyRuntimeClass myrc2{ nullptr };
myrc2 = winrt::make<MyProject::implementation::MyRuntimeClass>();
```

- You can access the members of all of the interfaces of a projected type.
- You can return a projected type to a caller.
- Projected types and interfaces derive from **winrt::Windows::Foundation::IUnknown**. So, you can call **IUnknown::as** on a projected type or interface to query for other projected interfaces, which you can also either use or return to a caller. The **as** member function works like **QueryInterface**.

C++/WinRT

```
void f(MyProject::MyRuntimeClass const& myrc)
{
    myrc.ToString();
    myrc.Close();
    IClosable iclosable = myrc.as<IClosable>();
    iclosable.Close();
}
```

# Activation factories

The convenient, direct way to create a C++/WinRT object is as follows.

C++/WinRT

```
using namespace winrt::Windows::Globalization::NumberFormatting;
...
```

```
CurrencyFormatter currency{ L"USD" };
```

But there may be times that you'll want to create the activation factory yourself, and then create objects from it at your convenience. Here are some examples showing you how, using the **winrt::get_activation_factory** function template.

C++/WinRT

```
using namespace winrt::Windows::Globalization::NumberFormatting;
...
auto factory = winrt::get_activation_factory<CurrencyFormatter,
ICurrencyFormatterFactory>();
CurrencyFormatter currency = factory.CreateCurrencyFormatterCode(L"USD");
```

C++/WinRT

```
using namespace winrt::Windows::Foundation;
...
auto factory = winrt::get_activation_factory<Uri, IUriRuntimeClassFactory>
();
Uri uri = factory.CreateUri(L"http://www.contoso.com");
```

The classes in the two examples above are types from a Windows namespace. In this next example, **ThermometerWRC::Thermometer** is a custom type implemented in a Windows Runtime component.

C++/WinRT

```
auto factory = winrt::get_activation_factory<ThermometerWRC::Thermometer>();
ThermometerWRC::Thermometer thermometer =
factory.ActivateInstance<ThermometerWRC::Thermometer>();
```

# Member/Type ambiguities

When a member function has the same name as a type, there's ambiguity. The rules for C++ unqualified name lookup in member functions cause it to search the class before searching in namespaces. The *substitution failure is not an error* (SFINAE) rule doesn't apply (it applies during overload resolution of function templates). So if the name inside the class doesn't make sense, then the compiler doesn't keep looking for a better match—it simply reports an error.

C++/WinRT

```
struct MyPage : Page
{
    void DoWork()
    {
        // This doesn't compile. You get the error
        // "'winrt::Windows::Foundation::IUnknown::as':
        // no matching overloaded function found".
        auto style{ Application::Current().Resources().
            Lookup(L"MyStyle").as<Style>() };
    }
}
```

Above, the compiler thinks that you're passing **FrameworkElement.Style()** (which, in C++/WinRT, is a member function) as the template parameter to **IUnknown::as**. The solution is to force the name `Style` to be interpreted as the type **Windows::UI::Xaml::Style**.

C++/WinRT

```
struct MyPage : Page
{
    void DoWork()
    {
        // One option is to fully-qualify it.
        auto style{ Application::Current().Resources().
            Lookup(L"MyStyle").as<Windows::UI::Xaml::Style>() };

        // Another is to force it to be interpreted as a struct name.
        auto style{ Application::Current().Resources().
            Lookup(L"MyStyle").as<struct Style>() };

        // If you have "using namespace Windows::UI;", then this is
sufficient.
        auto style{ Application::Current().Resources().
            Lookup(L"MyStyle").as<Xaml::Style>() };

        // Or you can force it to be resolved in the global namespace (into
which
        // you imported the Windows::UI::Xaml namespace when you did
        // "using namespace Windows::UI::Xaml;".
        auto style = Application::Current().Resources().
            Lookup(L"MyStyle").as<::Style>();
    }
}
```

Unqualified name lookup has a special exception in the case that the name is followed by `::`, in which case it ignores functions, variables, and enum values. This allows you to do things like this.

C++/WinRT

```
struct MyPage : Page
{
    void DoSomething()
    {
        Visibility(Visibility::Collapsed); // No ambiguity here (special
exception).
    }
}
```

The call to `Visibility()` resolves to the **UIElement.Visibility** member function name. But the parameter `Visibility::Collapsed` follows the word `Visibility` with `::`, and so the method name is ignored, and the compiler finds the enum class.

# Important APIs

- QueryInterface function
- RoActivateInstance function
- Windows::Foundation::Uri class
- winrt::get_activation_factory function template
- winrt::make function template
- winrt::Windows::Foundation::IUnknown struct

# Related topics

- Author events in C++/WinRT
- Interop between C++/WinRT and the ABI
- Introduction to C++/WinRT
- Windows Runtime components with C++/WinRT
- XAML controls; bind to a C++/WinRT property

# Feedback

# Author APIs with C++/WinRT

Article • 12/18/2023

This topic shows how to author C++/WinRT APIs by using the **winrt::implements** base struct, either directly or indirectly. Synonyms for *author* in this context are *produce*, or *implement*. This topic covers the following scenarios for implementing APIs on a C++/WinRT type, in this order.

> **Note**
>
> This topic touches on the subject of Windows Runtime components, but only in the context of C++/WinRT. If you're looking for content about Windows Runtime components that covers all Windows Runtime languages, then see Windows Runtime components.

- You're *not* authoring a Windows Runtime class (runtime class); you just want to implement one or more Windows Runtime interfaces for local consumption within your app. You derive directly from **winrt::implements** in this case, and implement functions.
- You *are* authoring a runtime class. You might be authoring a component to be consumed from an app. Or you might be authoring a type to be consumed from XAML user interface (UI), and in that case you're both implementing and consuming a runtime class within the same compilation unit. In these cases, you let the tools generate classes for you that derive from **winrt::implements**.

In both cases, the type that implements your C++/WinRT APIs is called the *implementation type*.

> **Important**
>
> It's important to distinguish the concept of an implementation type from that of a projected type. The projected type is described in Consume APIs with C++/WinRT.

## If you're *not* authoring a runtime class

The simplest scenario is where your type is implementing a Windows Runtime interface, and you'll be consuming that type within the same app. In that case, your type doesn't need to be a runtime class; just an ordinary C++ class. For example, you might be writing an app based around CoreApplication.

If your type is referenced by XAML UI, then it *does* need to be a runtime class, *even though it's in the same project* as the XAML. For that case, see the section If you're authoring a runtime class to be referenced in your XAML UI.

> **Note**
>
> For info about installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support), see Visual Studio support for C++/WinRT.

In Visual Studio, the **Visual C++** > **Windows Universal** > **Core App (C++/WinRT)** project template illustrates the **CoreApplication** pattern. The pattern begins with passing an implementation of **Windows::ApplicationModel::Core::IFrameworkViewSource** to **CoreApplication::Run**.

C++/WinRT

```
using namespace Windows::ApplicationModel::Core;
int __stdcall wWinMain(HINSTANCE, HINSTANCE, PWSTR, int)
{
    IFrameworkViewSource source = ...
    CoreApplication::Run(source);
}
```

**CoreApplication** uses the interface to create the app's first view. Conceptually, **IFrameworkViewSource** looks like this.

C++/WinRT

```
struct IFrameworkViewSource : IInspectable
{
    IFrameworkView CreateView();
};
```

Again conceptually, the implementation of **CoreApplication::Run** does this.

C++/WinRT

```
void Run(IFrameworkViewSource viewSource) const
{
    IFrameworkView view = viewSource.CreateView();
    ...
}
```

So you, as the developer, implement the **IFrameworkViewSource** interface. C++/WinRT has the base struct template **winrt::implements** to make it easy to implement an interface (or several) without resorting to COM-style programming. You just derive your type from **implements**, and then implement the interface's functions. Here's how.

C++/WinRT

```
// App.cpp
...
struct App : implements<App, IFrameworkViewSource>
{
    IFrameworkView CreateView()
    {
        return ...
    }
}
...
```

That's taken care of **IFrameworkViewSource**. The next step is to return an object that implements the **IFrameworkView** interface. You can choose to implement that interface on **App**, too. This next code example represents a minimal app that will at least get a window up and running on the desktop.

C++/WinRT

```
// App.cpp
...
struct App : implements<App, IFrameworkViewSource, IFrameworkView>
{
    IFrameworkView CreateView()
    {
        return *this;
    }

    void Initialize(CoreApplicationView const &) {}

    void Load(hstring const&) {}

    void Run()
    {
        CoreWindow window = CoreWindow::GetForCurrentThread();
        window.Activate();

        CoreDispatcher dispatcher = window.Dispatcher();
        dispatcher.ProcessEvents(CoreProcessEventsOption::ProcessUntilQuit);
    }

    void SetWindow(CoreWindow const & window)
    {
        // Prepare app visuals here
```

```
    }

    void Uninitialize() {}
};
...
```

Since your **App** type *is an* **IFrameworkViewSource**, you can just pass one to **Run**.

C++/WinRT

```
using namespace Windows::ApplicationModel::Core;
int __stdcall wWinMain(HINSTANCE, HINSTANCE, PWSTR, int)
{
    CoreApplication::Run(winrt::make<App>());
}
```

# If you're authoring a runtime class in a Windows Runtime component

If your type is packaged in a Windows Runtime component for consumption from another binary (the other binary is usually an application), then your type needs to be a runtime class. You declare a runtime class in a Microsoft Interface Definition Language (IDL) (.idl) file (see Factoring runtime classes into Midl files (.idl)).

Each IDL file results in a `.winmd` file, and Visual Studio merges all of those into a single file with the same name as your root namespace. That final `.winmd` file will be the one that the consumers of your component will reference.

Here's an example of declaring a runtime class in an IDL file.

idl

```
// MyRuntimeClass.idl
namespace MyProject
{
    runtimeclass MyRuntimeClass
    {
        // Declaring a constructor (or constructors) in the IDL causes the
runtime class to be
        // activatable from outside the compilation unit.
        MyRuntimeClass();
        String Name;
    }
}
```

This IDL declares a Windows Runtime (runtime) class. A runtime class is a type that can be activated and consumed via modern COM interfaces, typically across executable boundaries. When you add an IDL file to your project, and build, the C++/WinRT toolchain (`midl.exe` and `cppwinrt.exe`) generate an implementation type for you. For an example of the IDL file workflow in action, see XAML controls; bind to a C++/WinRT property.

Using the example IDL above, the implementation type is a C++ struct stub named **winrt::MyProject::implementation::MyRuntimeClass** in source code files named `\MyProject\MyProject\Generated Files\sources\MyRuntimeClass.h` and `MyRuntimeClass.cpp`.

The implementation type looks like this.

C++/WinRT

```
// MyRuntimeClass.h
...
namespace winrt::MyProject::implementation
{
    struct MyRuntimeClass : MyRuntimeClassT<MyRuntimeClass>
    {
        MyRuntimeClass() = default;

        winrt::hstring Name();
        void Name(winrt::hstring const& value);
    };
}

// winrt::MyProject::factory_implementation::MyRuntimeClass is here, too.
```

Notice that the F-bound polymorphism pattern being used (**MyRuntimeClass** uses itself as a template argument to its base, **MyRuntimeClassT**). This is also called the curiously recurring template pattern (CRTP). If you follow the inheritance chain upwards, you'll come across **MyRuntimeClass_base**.

You can simplify the implementation of simple properties by using Windows Implementation Libraries (WIL). Here's how:

C++/WinRT

```
// MyRuntimeClass.h
...
namespace winrt::MyProject::implementation
{
    struct MyRuntimeClass : MyRuntimeClassT<MyRuntimeClass>
    {
```

```
        MyRuntimeClass() = default;

        wil::single_threaded_rw_property<winrt::hstring> Name;
    };
}
```

See Simple properties.

C++/WinRT

```
template <typename D, typename... I>
struct MyRuntimeClass_base : implements<D, MyProject::IMyRuntimeClass, I...>
```

So, in this scenario, at the root of the inheritance hierarchy is the **winrt::implements** base struct template once again.

For more details, code, and a walkthrough of authoring APIs in a Windows Runtime component, see Windows Runtime components with C++/WinRT and Author events in C++/WinRT.

# If you're authoring a runtime class to be referenced in your XAML UI

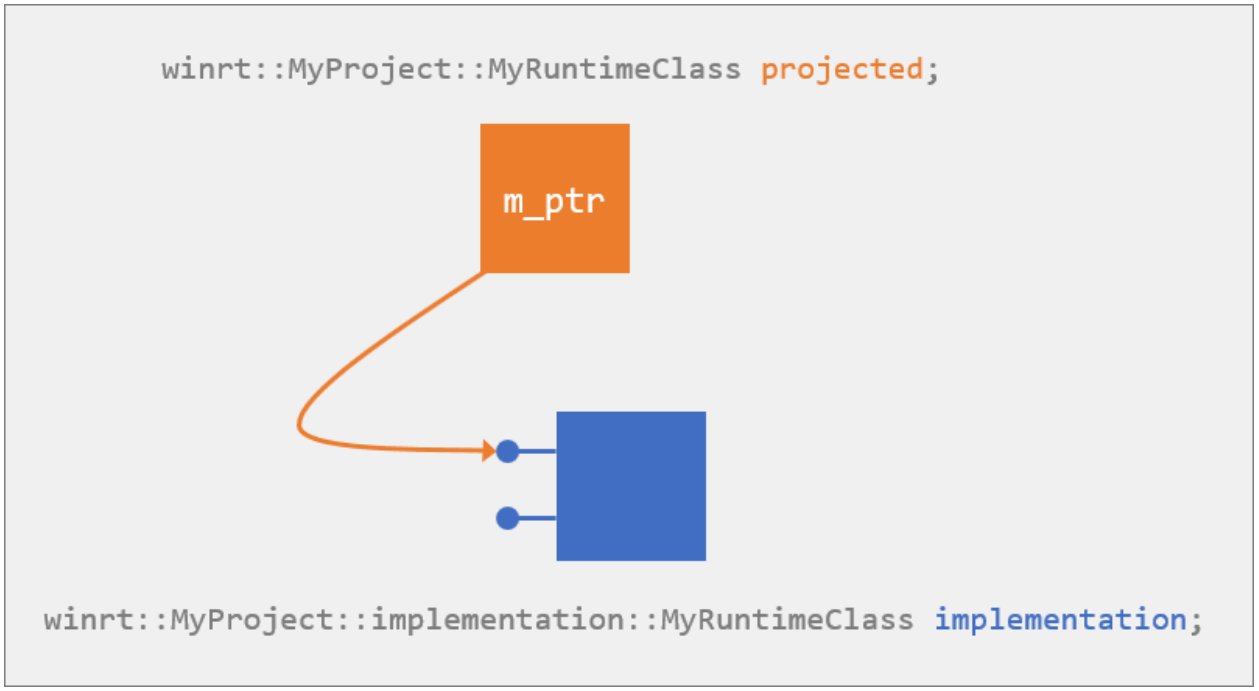
If your type is referenced by your XAML UI, then it needs to be a runtime class, even though it's in the same project as the XAML. Although they are typically activated across executable boundaries, a runtime class can instead be used within the compilation unit that implements it.

In this scenario, you're both authoring *and* consuming the APIs. The procedure for implementing your runtime class is essentially the same as that for a Windows Runtime component. So, see the previous section—If you're authoring a runtime class in a Windows Runtime component. The only detail that differs is that, from the IDL, the C++/WinRT toolchain generates not only an implementation type but also a projected type. It's important to appreciate that saying only "**MyRuntimeClass**" in this scenario may be ambiguous; there are several entities with that name, of different kinds.

- **MyRuntimeClass** is the name of a runtime class. But this is really an abstraction: declared in IDL, and implemented in some programming language.
- **MyRuntimeClass** is the name of the C++ struct **winrt::MyProject::implementation::MyRuntimeClass**, which is the C++/WinRT implementation of the runtime class. As we've seen, if there are separate implementing and consuming projects, then this struct exists only in the

implementing project. This is *the implementation type*, or *the implementation*. This type is generated (by the `cppwinrt.exe` tool) in the files `\MyProject\MyProject\Generated Files\sources\MyRuntimeClass.h` and `MyRuntimeClass.cpp`.

- **MyRuntimeClass** is the name of the projected type in the form of the C++ struct **winrt::MyProject::MyRuntimeClass**. If there are separate implementing and consuming projects, then this struct exists only in the consuming project. This is *the projected type*, or *the projection*. This type is generated (by `cppwinrt.exe`) in the file `\MyProject\MyProject\Generated Files\winrt\impl\MyProject.2.h`.




Here are the parts of the projected type that are relevant to this topic.

C++/WinRT

```
// MyProject.2.h
...
namespace winrt::MyProject
{
    struct MyRuntimeClass : MyProject::IMyRuntimeClass
    {
        MyRuntimeClass(std::nullptr_t) noexcept {}
        MyRuntimeClass();
    };
}
```

For an example walkthrough of implementing the **INotifyPropertyChanged** interface on a runtime class, see XAML controls; bind to a C++/WinRT property.

The procedure for consuming your runtime class in this scenario is described in Consume APIs with C++/WinRT.

# Factoring runtime classes into Midl files (.idl)

The Visual Studio project and item templates produce a separate IDL file for each runtime class. That gives a logical correspondence between an IDL file and its generated source code files.

However, if you consolidate all of your project's runtime classes into a single IDL file, then that can significantly improve build time. If you would otherwise have complex (or circular) `import` dependencies among them, then consolidating may actually be necessary. And you may find it easier to author and review your runtime classes if they're together.

# Runtime class constructors

Here are some points to take away from the listings we've seen above.

- Each constructor you declare in your IDL causes a constructor to be generated on both your implementation type and on your projected type. IDL-declared constructors are used to consume the runtime class from *a different* compilation unit.
- Whether you have IDL-declared constructor(s) or not, a constructor overload that takes **std::nullptr_t** is generated on your projected type. Calling the **std::nullptr_t** constructor is *the first of two steps* in consuming the runtime class from *the same* compilation unit. For more details, and a code example, see Consume APIs with C++/WinRT.
- If you're consuming the runtime class from *the same* compilation unit, then you can also implement non-default constructors directly on the implementation type (which, remember, is in `MyRuntimeClass.h`).

> **Note**
>
> If you expect your runtime class to be consumed from a different compilation unit (which is common), then include constructor(s) in your IDL (at least a default constructor). By doing that, you'll also get a factory implementation alongside your implementation type.
>
> If you want to author and consume your runtime class only within the same compilation unit, then don't declare any constructor(s) in your IDL. You don't need

a factory implementation, and one won't be generated. Your implementation type's default constructor will be deleted, but you can easily edit it and default it instead.

If you want to author and consume your runtime class only within the same compilation unit, and you need constructor parameters, then author the constructor(s) that you need directly on your implementation type.

## Runtime class methods, properties, and events

We've seen that the workflow is to use IDL to declare your runtime class and its members, and then the tooling generates prototypes and stub implementations for you. As for those autogenerated prototypes for the members of your runtime class, you *can* edit them so that they pass around different types from the types that you declare in your IDL. But you can do that only as long as the type that you declare in IDL can be forwarded to the type that you declare in the implemented version.

Here are some examples.

- You can relax parameter types. For example, if in IDL your method takes a **SomeClass**, then you could choose to change that to **IInspectable** in your implementation. This works because any **SomeClass** can be forwarded to **IInspectable** (the reverse, of course, wouldn't work).
- You can accept a copyable parameter by value, instead of by reference. For example, change `SomeClass const&` to `SomeClass`. That's necessary when you need to avoid capturing a reference into a coroutine (see Parameter-passing).
- You can relax the return value. For example, you can change **void** to **winrt::fire_and_forget**.

The last two are very useful when you're writing an asynchronous event handler.

## Instantiating and returning implementation types and interfaces

For this section, let's take as an example an implementation type named **MyType**, which implements the **IStringable** and **IClosable** interfaces.

You can derive **MyType** directly from **winrt::implements** (it's not a runtime class).

C++/WinRT

```
#include <winrt/Windows.Foundation.h>

using namespace winrt;
using namespace Windows::Foundation;

struct MyType : implements<MyType, IStringable, IClosable>
{
    winrt::hstring ToString(){ ... }
    void Close(){}
};
```

Or you can generate it from IDL (it's a runtime class).

idl

```
// MyType.idl
namespace MyProject
{
    runtimeclass MyType: Windows.Foundation.IStringable,
Windows.Foundation.IClosable
    {
        MyType();
    }
}
```

You can't directly allocate your implementation type.

C++/WinRT

```
MyType myimpl; // error C2259: 'MyType': cannot instantiate abstract class
```

But you can go from **MyType** to an **IStringable** or **IClosable** object that you can use or return as part of your projection by calling the **winrt::make** function template. **make** returns the implementation type's default interface.

C++/WinRT

```
IStringable istringable = winrt::make<MyType>();
```

> **Note**
>
> However, if you're referencing your type from your XAML UI, then there will be both an implementation type and a projected type in the same project. In that case,

**make** returns an instance of the projected type. For a code example of that scenario, see XAML controls; bind to a C++/WinRT property.

We can use `istringable` (in the code example above) only to call the members of the **IStringable** interface. But a C++/WinRT interface (which is a projected interface) derives from **winrt::Windows::Foundation::IUnknown**. So, you can call **IUnknown::as** (or **IUnknown::try_as**) on it to query for other projected types or interfaces, which you can also either use or return.

> **Tip**
>
> A scenario where you *shouldn't* call **as** or **try_as** is runtime class derivation ("composable classes"). When an implementation type composes another class, don't call **as** or **try_as** in order to perform an unchecked or checked **QueryInterface** of the class being composed. Instead, access the (`this->`) `m_inner` data member, and call **as** or **try_as** on that. For more info, see **Runtime class derivation** in this topic.

C++/WinRT

```
istringable.ToString();
IClosable iclosable = istringable.as<IClosable>();
iclosable.Close();
```

If you need to access all of the implementation's members, and then later return an interface to a caller, then use the **winrt::make_self** function template. **make_self** returns a **winrt::com_ptr** wrapping the implementation type. You can access the members of all of its interfaces (using the arrow operator), you can return it to a caller as-is, or you can call **as** on it and return the resulting interface object to a caller.

C++/WinRT

```
winrt::com_ptr<MyType> myimpl = winrt::make_self<MyType>();
myimpl->ToString();
myimpl->Close();
IClosable iclosable = myimpl.as<IClosable>();
iclosable.Close();
```

The **MyType** class is not part of the projection; it's the implementation. But this way you can call its implementation methods directly, without the overhead of a virtual function call. In the example above, even though **MyType::ToString** uses the same signature as the projected method on **IStringable**, we're calling the non-virtual method directly,

without crossing the application binary interface (ABI). The **com_ptr** simply holds a pointer to the **MyType** struct, so you can also access any other internal details of **MyType** via the `myimpl` variable and the arrow operator.

In the case where you have an interface object, and you happen to know that it's an interface on your implementation, then you can get back to the implementation using the **winrt::get_self** function template. Again, it's a technique that avoids virtual function calls, and lets you get directly at the implementation.

> ⓘ **Note**
>
> If you haven't installed the Windows SDK version 10.0.17763.0 (Windows 10, version 1809), or later, then you need to call winrt::from_abi instead of winrt::get_self.

Here's an example. There's another example in Implement the **BgLabelControl** custom control class.

C++/WinRT

```
void ImplFromIClosable(IClosable const& from)
{
    MyType* myimpl = winrt::get_self<MyType>(from);
    myimpl->ToString();
    myimpl->Close();
}
```

But only the original interface object holds on to a reference. If *you* want to hold on to it, then you can call **com_ptr::copy_from**.

C++/WinRT

```
winrt::com_ptr<MyType> impl;
impl.copy_from(winrt::get_self<MyType>(from));
// com_ptr::copy_from ensures that AddRef is called.
```

The implementation type itself doesn't derive from **winrt::Windows::Foundation::IUnknown**, so it has no **as** function. Even so, as you can see in the **ImplFromIClosable** function above, you can access the members of all of its interfaces. But if you do that, then don't return the raw implementation type instance to the caller. Instead, use one of the techniques already shown, and return a projected interface, or a **com_ptr**.

If you have an instance of your implementation type, and you need to pass it to a function that expects the corresponding projected type, then you can do so, as shown in the code example below. A conversion operator exists on your implementation type (provided that the implementation type was generated by the `cppwinrt.exe` tool) that makes this possible. You can pass an implementation type value directly to a method that expects a value of the corresponding projected type. From an implementation type member function, you can pass `*this` to a method that expects a value of the corresponding projected type.

C++/WinRT

```
// MyClass.idl
import "MyOtherClass.idl";
namespace MyProject
{
    runtimeclass MyClass
    {
        MyClass();
        void MemberFunction(MyOtherClass oc);
    }
}

// MyClass.h
...
namespace winrt::MyProject::implementation
{
    struct MyClass : MyClassT<MyClass>
    {
        MyClass() = default;
        void MemberFunction(MyProject::MyOtherClass const& oc) {
oc.DoWork(*this); }
    };
}
...

// MyOtherClass.idl
import "MyClass.idl";
namespace MyProject
{
    runtimeclass MyOtherClass
    {
        MyOtherClass();
        void DoWork(MyClass c);
    }
}

// MyOtherClass.h
...
namespace winrt::MyProject::implementation
{
    struct MyOtherClass : MyOtherClassT<MyOtherClass>
```

```
    {
        MyOtherClass() = default;
        void DoWork(MyProject::MyClass const& c){ /* ... */ }
    };
}
...

//main.cpp
#include "pch.h"
#include <winrt/base.h>
#include "MyClass.h"
#include "MyOtherClass.h"
using namespace winrt;

// MyProject::MyClass is the projected type; the implementation type would
be MyProject::implementation::MyClass.

void FreeFunction(MyProject::MyOtherClass const& oc)
{
    auto defaultInterface = winrt::make<MyProject::implementation::MyClass>
();
    MyProject::implementation::MyClass* myimpl =
winrt::get_self<MyProject::implementation::MyClass>(defaultInterface);
    oc.DoWork(*myimpl);
}
...
```

# Runtime class derivation

You can create a runtime class that derives from another runtime class, provided that the base class is declared as "unsealed". The Windows Runtime term for class derivation is "composable classes". The code for implementing a derived class depends on whether the base class is provided by another component or by the same component. Fortunately, you don't have to learn these rules—you can just copy the sample implementations from the `sources` output folder produced by the `cppwinrt.exe` compiler.

Consider this example.

idl

```
// MyProject.idl
namespace MyProject
{
    [default_interface]
    runtimeclass MyButton : Windows.UI.Xaml.Controls.Button
    {
        MyButton();
    }
```

```
    unsealed runtimeclass MyBase
    {
        MyBase();
        overridable Int32 MethodOverride();
    }

    [default_interface]
    runtimeclass MyDerived : MyBase
    {
        MyDerived();
    }
}
```

In the above example, **MyButton** is derived from the XAML **Button** control, which is provided by another component. In that case, the implementation looks just like the implementation of a non-composable class:

C++

```
namespace winrt::MyProject::implementation
{
    struct MyButton : MyButtonT<MyButton>
    {
    };
}

namespace winrt::MyProject::factory_implementation
{
    struct MyButton : MyButtonT<MyButton, implementation::MyButton>
    {
    };
}
```

On the other hand, in the above example, **MyDerived** is derived from another class in the same component. In this case, the implementation requires an additional template parameter specifying the implementation class for the base class.

C++

```
namespace winrt::MyProject::implementation
{
    struct MyDerived : MyDerivedT<MyDerived, implementation::MyBase>
    {                                       // ^^^^^^^^^^^^^^^^^^^^^^
    };
}

namespace winrt::MyProject::factory_implementation
{
    struct MyDerived : MyDerivedT<MyDerived, implementation::MyDerived>
```

```
    {
    };
}
```

In either case, your implementation can call a method from the base class by qualifying it with the `base_type` type alias:

C++

```
namespace winrt::MyProject::implementation
{
    struct MyButton : MyButtonT<MyButton>
    {
        void OnApplyTemplate()
        {
            // Call base class method
            base_type::OnApplyTemplate();

            // Do more work after the base class method is done
            DoAdditionalWork();
        }
    };

    struct MyDerived : MyDerivedT<MyDerived, implementation::MyBase>
    {
        int MethodOverride()
        {
            // Return double what the base class returns
            return 2 * base_type::MethodOverride();
        }
    };
}
```

**Tip**

When an implementation type composes another class, don't call **as** or **try_as** in order to perform an unchecked or checked **QueryInterface** of the class being composed. Instead, access the (`this->`) `m_inner` data member, and call **as** or **try_as** on that.

# Deriving from a type that has a non-default constructor

ToggleButtonAutomationPeer::ToggleButtonAutomationPeer(ToggleButton) is an example of a non-default constructor. There's no default constructor so, to construct a

**ToggleButtonAutomationPeer**, you need to pass an *owner*. Consequently, if you derive from **ToggleButtonAutomationPeer**, then you need to provide a constructor that takes an *owner* and passes it to the base. Let's see what that looks like in practice.

idl

```
// MySpecializedToggleButton.idl
namespace MyNamespace
{
    runtimeclass MySpecializedToggleButton :
        Windows.UI.Xaml.Controls.Primitives.ToggleButton
    {
        ...
    };
}
```

idl

```
// MySpecializedToggleButtonAutomationPeer.idl
namespace MyNamespace
{
    runtimeclass MySpecializedToggleButtonAutomationPeer :
        Windows.UI.Xaml.Automation.Peers.ToggleButtonAutomationPeer
    {
        MySpecializedToggleButtonAutomationPeer(MySpecializedToggleButton
owner);
    };
}
```

The generated constructor for your implementation type looks like this.

C++/WinRT

```
// MySpecializedToggleButtonAutomationPeer.cpp
...
MySpecializedToggleButtonAutomationPeer::MySpecializedToggleButtonAutomation
Peer
    (MyNamespace::MySpecializedToggleButton const& owner)
{
    ...
}
...
```

The only piece missing is that you need to pass that constructor parameter on to the base class. Remember the F-bound polymorphism pattern that we mentioned above? Once you're familiar with the details of that pattern as used by C++/WinRT, you can figure out what your base class is called (or you can just look in your implementation class's header file). This is how to call the base class constructor in this case.

C++/WinRT

```
// MySpecializedToggleButtonAutomationPeer.cpp
...
MySpecializedToggleButtonAutomationPeer::MySpecializedToggleButtonAutomation
Peer
    (MyNamespace::MySpecializedToggleButton const& owner) :

MySpecializedToggleButtonAutomationPeerT<MySpecializedToggleButtonAutomation
Peer>(owner)
{
    ...
}
...
```

The base class constructor expects a **ToggleButton**. And **MySpecializedToggleButton** *is a* **ToggleButton**.

Until you make the edit described above (to pass that constructor parameter on to the base class), the compiler will flag your constructor and point out that there's no appropriate default constructor available on a type called (in this case) **MySpecializedToggleButtonAutomationPeer_base<MySpecializedToggleButtonAutomationPeer>**. That's actually the base class of the bass class of your implementation type.

# Namespaces: projected types, implementation types, and factories

As you've seen previously in this topic, a C++/WinRT runtime class exists in the form of more than one C++ class in more than one namespace. So, the name **MyRuntimeClass** has one meaning in the **winrt::MyProject** namespace, and a different meaning in the **winrt::MyProject::implementation** namespace. Be aware of which namespace you currently have in context, and then use namespace prefixes if you need a name from a different namespace. Let's look more closely at the namespaces in question.

- **winrt::MyProject**. This namespace contains projected types. An object of a projected type is a proxy; it's essentially a smart pointer to a backing object, where that backing object might be implemented here in your project, or it might be implemented in another compilation unit.
- **winrt::MyProject::implementation**. This namespace contains implementation types. An object of an implementation type is not a pointer; it's a value—a full C++ stack object. Don't construct an implementation type directly; instead, call winrt::make, passing your implementation type as the template parameter. We've shown examples of **winrt::make** in action previously in this topic, and there's

another example in XAML controls; bind to a C++/WinRT property. Also see Diagnosing direct allocations.

- **winrt::MyProject::factory_implementation**. This namespace contains factories. An object in this namespace supports **IActivationFactory**.

This table shows the minimum namespace qualification you need to use in different contexts.

Expand table

| The namespace that's in context | To specify the projected type | To specify the implementation type |
|---|---|---|
| **winrt::MyProject** | `MyRuntimeClass` | `implementation::MyRuntimeClass` |
| **winrt::MyProject::implementation** | `MyProject::MyRuntimeClass` | `MyRuntimeClass` |

> **Important**
>
> When you want to return a projected type from your implementation, be careful not to instantiate the implementation type by writing `MyRuntimeClass myRuntimeClass;`. The correct techniques and code for that scenario are shown previously in this topic in the section Instantiating and returning implementation types and interfaces.
>
> The problem with `MyRuntimeClass myRuntimeClass;` in that scenario is that it creates a **winrt::MyProject::implementation::MyRuntimeClass** object on the stack. That object (of implementation type) behaves like the projected type in some ways—you can invoke methods on it in the same way; and it even converts to a projected type. But the object destructs, as per normal C++ rules, when the scope exits. So, if you returned a projected type (a smart pointer) to that object, then that pointer is now dangling.
>
> This memory corruption type of bug is difficult to diagnose. So, for debug builds, a C++/WinRT assertion helps you catch this mistake, using a stack-detector. But coroutines are allocated on the heap, so you won't get help with this mistake if you make it inside a coroutine. For more info, see Diagnosing direct allocations.

# Using projected types and implementation types with various C++/WinRT features

Here are various places where a C++/WinRT features expects a type, and what kind of type it expects (projected type, implementation type, or both).

Expand table

| Feature | Accepts | Notes |
|---|---|---|
| `T` (representing a smart pointer) | Projected | See the caution in Namespaces: projected types, implementation types, and factories about using the implementation type by mistake. |
| `agile_ref<T>` | Both | If you use the implementation type, then the constructor argument must be `com_ptr<T>`. |
| `com_ptr<T>` | Implementation | Using the projected type generates the error: `'Release' is not a member of 'T'`. |
| `default_interface<T>` | Both | If you use the implementation type, then the first implemented interface is returned. |
| `get_self<T>` | Implementation | Using the projected type generates the error: `'_abi_TrustLevel': is not a member of 'T'`. |
| `guid_of<T>()` | Both | Returns the GUID of the default interface. |
| `IWinRTTemplateInterface<T>` | Projected | Using the implementation type compiles, but it's a mistake—see the caution in Namespaces: projected types, implementation types, and factories. |
| `make<T>` | Implementation | Using the projected type generates the error: `'implements_type': is not a member of any direct or indirect base class of 'T'` |
| `make_agile(T const&amp;)` | Both | If you use the implementation type, then the argument must be `com_ptr<T>`. |
| `make_self<T>` | Implementation | Using the projected type generates the error: `'Release': is not a member of any direct or indirect base class of 'T'` |
| `name_of<T>` | Projected | If you use the implementation type, then you get the stringified GUID of the default interface. |
| `weak_ref<T>` | Both | If you use the implementation type, then the constructor argument must be `com_ptr<T>`. |

# Opt in to uniform construction, and direct implementation access

This section describes a C++/WinRT 2.0 feature that's opt-in, although it is enabled by default for new projects. For an existing project, you'll need to opt in by configuring the `cppwinrt.exe` tool. In Visual Studio, set project property **Common Properties** > **C++/WinRT** > **Optimized** to *Yes*. That has the effect of adding `<CppWinRTOptimized>true</CppWinRTOptimized>` to your project file. And it has the same effect as adding the switch when invoking `cppwinrt.exe` from the command line.

The `-opt[imize]` switch enables what's often called *uniform construction*. With uniform (or *unified*) construction, you use the C++/WinRT language projection itself to create and use your implementation types (types implemented by your component, for consumption by applications) efficiently and without any loader difficulties.

Before describing the feature, let's first show the situation *without* uniform construction. To illustrate, we'll begin with this example Windows Runtime class.

idl

```
// MyClass.idl
namespace MyProject
{
    runtimeclass MyClass
    {
        MyClass();
        void Method();
        static void StaticMethod();
    }
}
```

As a C++ developer familiar with using the C++/WinRT library, you might want to use the class like this.

C++/WinRT

```
using namespace winrt::MyProject;

MyClass c;
c.Method();
MyClass::StaticMethod();
```

And that would be perfectly reasonable provided that the consuming code shown didn't reside within the same component that implements this class. As a language projection, C++/WinRT shields you as a developer from the ABI (the COM-based application binary

interface that the Windows Runtime defines). C++/WinRT doesn't call directly into the implementation; it travels through the ABI.

Consequently, on the line of code where you're constructing a **MyClass** object (`MyClass c;`), the C++/WinRT projection calls **RoGetActivationFactory** to retrieve the class or activation factory, and then uses that factory to create the object. The last line likewise uses the factory to make what appears to be a static method call. All of this requires that your class be registered, and that your module implements the **DllGetActivationFactory** entry point. C++/WinRT has a very fast factory cache, so none of this causes a problem for an application consuming your component. The trouble is that, within your component, you've just done something that's a little problematic.

Firstly, no matter how fast the C++/WinRT factory cache is, calling through **RoGetActivationFactory** (or even subsequent calls through the factory cache) will always be slower than calling directly into the implementation. A call to **RoGetActivationFactory** followed by **IActivationFactory::ActivateInstance** followed by **QueryInterface** is obviously not going to be as efficient as using a C++ `new` expression for a locally-defined type. As a consequence, seasoned C++/WinRT developers are accustomed to using the **winrt::make** or **winrt::make_self** helper functions when creating objects within a component.

C++/WinRT

```
// MyClass c;
MyProject::MyClass c{ winrt::make<implementation::MyClass>() };
```

But, as you can see, that's not nearly as convenient nor concise. You must use a helper function to create the object, and you must also disambiguate between the implementation type and the projected type.

Secondly, using the projection to create the class means that its activation factory will be cached. Normally, that's what you want, but if the factory resides in the same module (DLL) that's making the call, then you've effectively pinned the DLL and prevented it from ever unloading. In many cases, that doesn't matter; but some system components *must* support unloading.

This is where the term *uniform construction* comes in. Regardless of whether creation code resides in a project that's merely consuming the class, or whether it resides in the project that's actually *implementing* the class, you can freely use the same syntax to create the object.

C++/WinRT

```
// MyProject::MyClass c{ winrt::make<implementation::MyClass>() };
MyClass c;
```

When you build your component project with the `-opt[imize]` switch, the call through the language projection compiles down to the same efficient call to the **winrt::make** function that directly creates the implementation type. That makes your syntax simple and predictable, it avoids any performance hit of calling through the factory, and it avoids pinning the component in the process. In addition to component projects, this is also useful for XAML applications. Bypassing **RoGetActivationFactory** for classes implemented in the same application allows you to construct them (without needing to be registered) in all the same ways you could if they were outside your component.

Uniform construction applies to *any* call that's served by the factory under the hood. Practically, that means that the optimization serves both constructors and static members. Here's that original example again.

C++/WinRT

```
MyClass c;
c.Method();
MyClass::StaticMethod();
```

Without `-opt[imize]`, the first and last statements require calls through the factory object. *With* `-opt[imize]`, neither of them do. And those calls are compiled directly against the implementation, and even have the potential to be inlined. Which speaks to the other term often used when talking about `-opt[imize]`, namely *direct implementation* access.

Language projections are convenient but, when you can directly access the implementation, you can and should take advantage of that to produce the most efficient code possible. C++/WinRT can do that for you, without forcing you to leave the safety and productivity of the projection.

This is a breaking change because the component must cooperate in order to allow the language projection to reach in and directly access its implementation types. As C++/WinRT is a header-only library, you can look inside and see what's going on. Without `-opt[imize]`, the **MyClass** constructor, and **StaticMethod** member, are defined by the projection like this.

C++/WinRT

```
namespace winrt::MyProject
{
```